

Week 8

Debugging: Follow the Evidence

Yongqiang Tian

Department of Software Systems and Cybersecurity
Faculty of IT, Monash University

Acknowledgement

I wish to acknowledge the people of the Kulin Nations, on whose land we are gathered today. I pay my respects to their Elders, past and present.

The class report is wrong

A tutor runs the weekly report

```
$ python3 grade_report.py \  
    data/class.csv  
Class average: 18.75
```

Bug report

“The average should be about 74.
The report is wrong.”

What evidence would you collect before changing the code?

Choose the next useful evidence

By the end of today, you should be able to:

- ① turn a vague failure into a **reproducible case**;
- ② choose between **static checks**, **logs**, and a **debugger**;
- ③ fix the demonstrated cause and **verify** that it stays fixed.

Debugging is not guessing. It is reducing uncertainty.

A complaint is not yet a reproduction

Too vague

“The average is wrong.”

Missing	Why it matters
Command and input	We may not run the same case.
Expected result	“Wrong” has no testable meaning.
Actual result	We cannot compare or track a change.
Environment	The failure may depend on where it ran.

Turn the complaint into instructions someone else can follow.

Activity 1: Reproduce the failure

Work individually

Complete the reproduction card in the handout for the grade-report failure.

Your card must name:

command input expected actual

Output

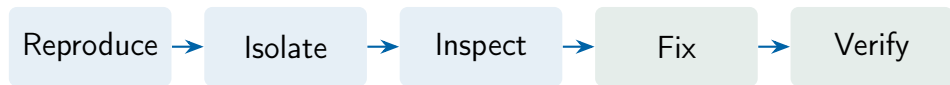
Another student should be able to reproduce the same failure without asking you a question.

You have just written a useful bug report

How to run it	command and environment
What to run it on	smallest relevant input
How to judge it	expected and actual behaviour
What supports it	concise logs, traces, or screenshots when relevant

A strong bug report lets someone reproduce, observe, and judge the same failure.

Debugging follows the evidence



Each step should answer one question and create the next one.

If an experiment does not distinguish between hypotheses, it is weak evidence.

Shrink the case, keep the failure

Full report

```
$ python3 grade_report.py \  
    data/class.csv  
Class average: 18.75
```

CSV parsing, names, and formatting are all present.

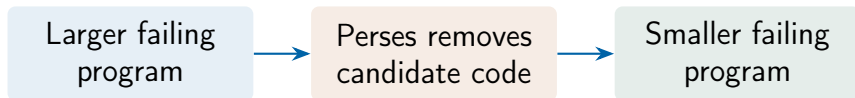
Small failing case

```
average([2, 4, 6])  
  
expected: 4  
actual:    2
```

Only the calculation remains.

A smaller reproduction removes explanations we no longer need.

Perses automates the same shrinking question



The keep-or-discard question

Does an executable test confirm that the *same* failure is still present?

The test script is an executable definition of the bug.

The program runs, but its state drifts

```
def average(values):  
    total = 0  
    for value in values:  
        total = value  
    return total / len(values)
```

Input

[2, 4, 6]

The function does not crash. The final answer is simply wrong.

At which iteration does the state first differ from what we expect?

Activity 2: Find the first wrong state

Work individually

Complete the three-iteration state table in the handout.

- 1 Predict the expected value of `total`.
- 2 Trace the actual value produced by the code.
- 3 Circle the first row where they differ.
- 4 Connect the faulty line to the visible failure.

Output

One completed table and one explanation of the first wrong state.

The RIP model explains how a fault becomes a failure

Fault in the code

```
total = value
```

R: Reachability

second iteration
executes the line

I: Infection

total becomes 4,
not 6

P: Propagation

wrong total reaches
the division

Visible failure

returns 2,
not 4

A fault may exist without producing a failure unless all three conditions occur.

Static checks are cheap, not complete

Static check

```
$ pylint average.py
```

```
Your code has been rated  
at 10.00/10
```

Behavioural test

```
$ python3 -m unittest
```

```
FAIL: expected 4,  
       received 2
```

Exact Pylint messages and scores vary by installed version.

Clean lint means “no detected static issue”, not “correct program”.

Activity 3: Choose the next evidence

Work individually

For each handout scenario, choose a static check, logging, or a debugger.

- 1 An undefined name may exist before execution.
- 2 A production job fails, but cannot be paused.
- 3 A local loop has a wrong intermediate value.
- 4 Lint passes, but a behavioural test fails.

Output

Four choices. For each one, state what the evidence would distinguish between.

Runtime evidence answers a different question

How does `total` change while the loop runs?

Cannot pause the program

Production, CI, a remote machine, or an intermittent failure.

Use targeted **logging**.

Can reproduce locally

Pause before the suspicious assignment and inspect changing state.

Use a **debugger**.

Use logs when you cannot pause

Record the state that tests the hypothesis

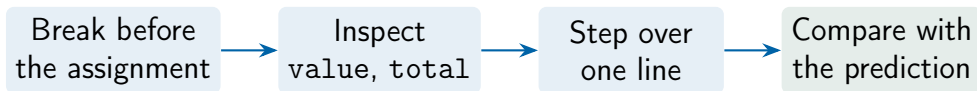
```
logger.debug(  
    "value=%s total=%s",  
    value, total)
```

Remote run

```
DEBUG value=2 total=2  
DEBUG value=4 total=4  
DEBUG value=6 total=6
```

Useful logs expose a decision or state change; they do not narrate every line.

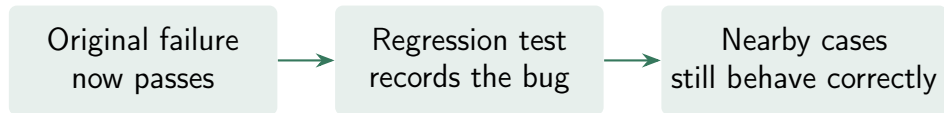
Use a debugger when the failure is local



VS Code and pdb are interfaces to the same investigation.

Breakpoint, step, inspect, and call stack are the transferable concepts.

One passing case is not enough



“Fixed” is a claim supported by repeatable evidence.

A larger project creates new uncertainty

Small example

- one function
- one machine
- one person
- repeatable input

Group project

- many components
- different environments
- several owners
- timing and shared state

The method survives; collecting connected evidence becomes harder.

Activity 4: Debugging at project scale

In groups of 3–4, choose one incident from the handout:

- 1 the same commit behaves differently on two laptops;
- 2 a CI test fails about once in every ten runs;
- 3 small data works, but the full dataset fails after 20 minutes;
- 4 import → validate → calculate → export produces a wrong file.

Do not solve the bug

Design the investigation that would distinguish between plausible causes.

Activity 4 output: An investigation plan

Your plan must answer six questions:

- ① What is the visible failure?
- ② Why is reproduction difficult?
- ③ Which boundary would you inspect first?
- ④ What evidence would you collect there?
- ⑤ Which two explanations would that evidence distinguish?
- ⑥ How would you verify the eventual fix?

One-minute report

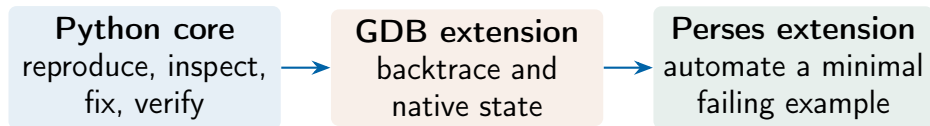
"We would collect _____ first, because it would distinguish between _____ and _____."

At scale, the workflow stays; the evidence spreads out

Small program	Group or large project
one machine	different environments
one execution path	many component boundaries
deterministic failure	timing and intermittent failures
one person knows the code	knowledge is distributed
one output stream	evidence must be correlated

Reproduce, isolate, inspect, fix, and verify still applies.

Applied turns the principles into tool practice



The language and interface change; the evidence questions do not.

What to remember

- ① Make the failure reproducible before changing code.
- ② Inspect the first wrong state, not only the final symptom.
- ③ Match the evidence tool to the execution context.
- ④ Verify the original case, a regression test, and nearby cases.

Next week

Week 8 establishes that the program is correct. Week 9 asks whether it is fast enough.